

ARGoS Wind / Air Resistance — Developer Manual

Roi Sela

October 2025

Repository: [roiSela/air-epuck2](#)

Contents

1	Audience and Scope	3
2	Repository Overview	3
3	ARGoS Engines Used by This Project	3
3.1	Physics engine: dynamics2d (Chipmunk)	3
3.2	Robot model plugins (e-puck2, foot-bot, others)	4
3.3	Visualization engine: Qt-OpenGL	4
4	Architecture	4
4.1	Robot-agnostic controller design	4
4.2	Core Controller: CAirResistance	4
5	Physics Integration Details	5
6	Configuration Interface	5
6.1	Global configuration	5
6.2	Units and meanings	6
6.3	Per-controller parameters	6
7	Algorithmic Details (with Equations)	6
7.1	Impulse Model (definition of $\mathbf{w}_{\text{eff}}$ + numeric example)	6
7.2	Wake Blocking Reduction	7
8	Implementation Map (Math $\rightarrow$ Code Locations)	8
9	RAB Sensor: Role, Contract, Alternatives	9
9.1	Role	9
9.2	Contract (byte-level)	9
9.3	Alternatives	9
10	Pseudocode: Core Functions	9
10.1	Init(t_{node})	9
10.2	EnsurePhysicsHandle()	10
10.3	HandleAerodynamicsPreStep()	10
10.4	ComputeEffectiveWind()	11
10.5	IsBlockedByRAB(out r)	11
10.6	DriveImpulse(v_{cm_s})	12
10.7	ApplyWindImpulse()	12
10.8	HandleAerodynamicsPostStep()	12
10.9	GetYawRadians()	12
11	Tuning and Debugging	12
12	Doxygen API Documentation	13
12.1	What Doxygen is for	13
12.2	How to rebuild	13
12.3	How to view	13

1 Audience and Scope

This guide targets developers who extend, maintain, or embed this project’s wind / air-resistance behavior in ARGoS 3. It covers: repository layout, the impulse pipeline, aerodynamic blocking via RAB, configuration schema (including wake tunables), the `dynamics2d`/`Chipmunk` integration point, and extension points for derived controllers. It also includes pseudocode aligned with the C++ implementation.

2 Repository Overview

Path	Purpose
<code>src/controllers/air_resistance/</code>	Base controller (CAirResistance): wind + drive impulse pipeline, RAB-based aerodynamic blocking (wake).
<code>examples/controllers/wind_aware/</code>	Example derived controller: crosswind compensation (“crabbing”) built on top of CAirResistance .
<code>src/controllers/formation_template/</code> (if enabled)	Example derived controller: multi-robot coordination template under wind while preserving the base aerodynamics pipeline.
<code>src/loop_functions/wind_loop_functions/</code>	Loop-functions plugin: reads global wind from XML, provides shared access for visualization, and other experiment-wide logic.
<code>src/loop_functions/wind_loop_functions/</code> (Qt user-functions)	Qt-OpenGL user-functions plugin that draws the wind vector arrow in the world view.
<code>examples/</code>	Runnable ARGoS experiment configurations (<code>.argos</code>) used in the user manual.
<code>doc/</code>	LaTeX manuals + Doxygen configuration (<code>Doxyfile</code>); generated HTML is not committed.

3 ARGoS Engines Used by This Project

ARGoS is modular: a configuration file selects which *plugins* are used for physics, robot models, visualization, and media.

3.1 Physics engine: `dynamics2d` (**Chipmunk**)

This project integrates at the **dynamics2d** physics level. In ARGoS 3, **dynamics2d** is a 2D physics-engine plugin whose rigid-body backend is **Chipmunk**. The wind/drive forces are applied as **Chipmunk impulses** to the underlying `cpBody` after each physics tick.

In the experiment XML, the physics engine is selected explicitly:

```
<physics_engines>
  <dynamics2d id="dyn2d"/>
</physics_engines>
```

Why “Chipmunk callbacks” appear in this codebase. `Chipmunk` provides a *post-step callback* mechanism to safely apply impulses after collision resolution. ARGoS `dynamics2d` exposes `Chipmunk` bodies internally; this project schedules one post-step callback per tick to apply the accumulated impulse vector (see Sec. 4.2).

What if a different physics engine is used? If the experiment selects a different physics engine (e.g., `dynamics3d`), there is no Chipmunk `cpBody`. Therefore, this project’s impulse integration is designed for `dynamics2d`. Experiments must include `dynamics2d` for the wind/impulse behavior to take effect.

3.2 Robot model plugins (e-puck2, foot-bot, others)

The controller itself is **robot-agnostic** at the ARGoS interface level (generic sensors/actuators, see Sec. 4.1), but the experiment chooses specific robot model plugins:

```
<framework>
  ...
</framework>

<!-- Example robot plugins loaded by ARGoS at runtime -->
<!-- (e-puck2, foot-bot, etc.) -->
```

3.3 Visualization engine: Qt-OpenGL

The wind arrow is drawn by a Qt user-functions plugin (optional). In XML it appears under:

```
<visualization>
  <qt-opengl>
    <user_functions library="..." label="wind_qt_user_functions"/>
  </qt-opengl>
</visualization>
```

4 Architecture

4.1 Robot-agnostic controller design

The base controller `CAirResistance` uses only **generic** ARGoS interfaces:

- `CCI_DifferentialSteeringActuator`
- `CCI_PositioningSensor`
- `CCI_RangeAndBearingSensor` / `Actuator`

No e-puck2-specific CCI interfaces are required by the core implementation. Robot specificity belongs in **experiment XML** (robot type tags, robot plugins to load), and in optional **example controllers**.

4.2 Core Controller: `CAirResistance`

Lifecycle and Extensibility. `CAirResistance` derives from `CCI_Controller`. Lifecycle methods and key helpers are **virtual** and designed for subclassing.

Important protected hooks (what they do).

- `EnsurePhysicsHandle()` — Cache the Chipmunk body pointer and derive the robot’s effective radius (meters) from the AAB. Safe to call every tick.
- `GetYawRadians()` — Return yaw ψ (world frame) from the positioning sensor; used for the forward unit vector $\hat{\mathbf{f}}(\psi)$.
- `HandleAerodynamicsPreStep()` — Reset $\mathbf{J}_{\text{sum}}$; add wind impulse using $\mathbf{w}_{\text{eff}}$; broadcast radius (RAB byte 0).

- `HandleAerodynamicsPostStep()` — Register a Chipmunk post-step callback to apply $\mathbf{J}_{\text{sum}}$ after collisions.
- `ComputeEffectiveWind()` — Compute $\mathbf{w}_{\text{eff}} = (1 - r)\mathbf{w}$ where r is the wake reduction from neighbors.
- `IsBlockedByRAB(Real&r)` — From RAB neighbors, compute $r \in [0, 1]$ via geometric overlap of wakes (Sec. 7.2).
- `DriveImpulse(Realv_cm_s)` — Add forward impulse $\frac{m}{100}v\hat{\mathbf{f}}(\psi)$ (cm/s $\rightarrow$ m/s factor 1/100).
- `ApplyWindImpulse()` — Add $\frac{m}{100}\mathbf{w}_{\text{eff}}$ to the per-tick accumulator.

Impulse Pipeline (per tick).

1. **Pre-step:** Reset the impulse accumulator; compute and add the wind impulse (possibly reduced by blocking); broadcast radius via RAB.
2. **Drive:** Add a forward drive impulse along current yaw using the desired base speed.
3. **Post-step:** Schedule a single Chipmunk *post-step* callback to apply the summed impulse after collisions.

AABB (Axis-Aligned Bounding Box). An **AABB** is the smallest rectangle (in world axes) that contains the robot body: it has min/max corners $(x_{\min}, y_{\min})$ and $(x_{\max}, y_{\max})$. We derive an effective radius R from its width/height:

$$R \approx \frac{1}{2} \max(x_{\max} - x_{\min}, y_{\max} - y_{\min}).$$

This gives a robust, robot-model-independent scale for wake geometry.

5 Physics Integration Details

This project integrates with ARGoS dynamics2d object models:

- **Single-body robots** (e.g., e-puck2) expose a single Chipmunk `cpBody`.
- **Multi-body robots** (e.g., foot-bot) expose multiple Chipmunk bodies; body index 0 is treated as the chassis.

The controller caches the `cpBody*` and uses it to apply impulses post-step at the center of mass (COM).

6 Configuration Interface

6.1 Global configuration

Under `<configuration>` declare wind and (optionally) wake tunables:

```
<configuration>
  <air_resistance
    angle_deg="0"
    magnitude="15.0"
    lateral_reach_radii="3.0"
    shadow_length_radii="4.0"
    gamma_boost="2.0"
    upwind_gate_radii="0.5"
    adv_radius_min_m="0.005"
```

```

    adv_radius_max_m="0.20"/>
</configuration>

```

6.2 Units and meanings

Attribute	Units	Meaning
angle_deg	degrees	Wind direction in world frame.
magnitude	cm/s	Wind speed magnitude.
lateral_reach_rad	radii	Wake width scale ($\sigma_{\perp} = \text{lateral_reach_rad} \cdot R$).
shadow_length_rad	radii	Wake length scale ($L = \text{shadow_length_rad} \cdot R$).
gamma_boost	unitless	Nonlinear emphasis of mid-range overlaps.
upwind_gate_rad	radii	Minimum along-wind gap before wake starts.
adv_radius_min_m	meters	Sanity min for advertised RAB radius.
adv_radius_max_m	meters	Sanity max for advertised RAB radius.

6.3 Per-controller parameters

Each controller instance can specify a desired base forward speed:

```
<params velocity="15.0"/>
```

Units: velocity is in cm/s.

7 Algorithmic Details (with Equations)

7.1 Impulse Model (definition of $\mathbf{w}_{\text{eff}}$ + numeric example)

Per tick we accumulate world-frame impulses:

$$\mathbf{J}_{\text{wind}} = \frac{m}{100} \underbrace{\mathbf{w}_{\text{eff}}}_{\text{effective wind in cm/s}}, \quad (1)$$

$$\mathbf{J}_{\text{drive}} = \frac{m}{100} v_d \hat{\mathbf{f}}(\psi), \quad (2)$$

$$\mathbf{J}_{\text{sum}} = \mathbf{J}_{\text{wind}} + \mathbf{J}_{\text{drive}}. \quad (3)$$

Definition. Let $\mathbf{w}$ be the global wind and $r \in [0, 1]$ the reduction due to upwind neighbors. The effective wind is

$$\mathbf{w}_{\text{eff}} = (1 - r) \mathbf{w} \quad (\text{with } r \text{ computed in Sec. 7.2}).$$

Here m is body mass (Chipmunk), v_d the desired forward speed (cm/s), and $\hat{\mathbf{f}}(\psi)$ the unit forward vector given yaw ψ . A single post-step callback applies $\mathbf{J}_{\text{sum}}$ at the COM after collisions.

Numeric example (with picture). Let $m = 0.08$ kg, $v_d = 10$ cm/s, and $\mathbf{w} = (20$ cm/s, 0). If $r = 0.4 \Rightarrow \mathbf{w}_{\text{eff}} = (12$ cm/s, 0). With yaw $\psi = 60^\circ$, $\hat{\mathbf{f}}(\psi) = (\cos 60^\circ, \sin 60^\circ) = (0.5, 0.866)$.

$$\mathbf{J}_{\text{wind}} = \frac{0.08}{100} (12, 0) = (0.0096, 0), \quad \mathbf{J}_{\text{drive}} = \frac{0.08}{100} \cdot 10 (0.5, 0.866) = (0.0040, 0.00693),$$

so $\mathbf{J}_{\text{sum}} \approx (0.0136, 0.00693)$ kg m/s.

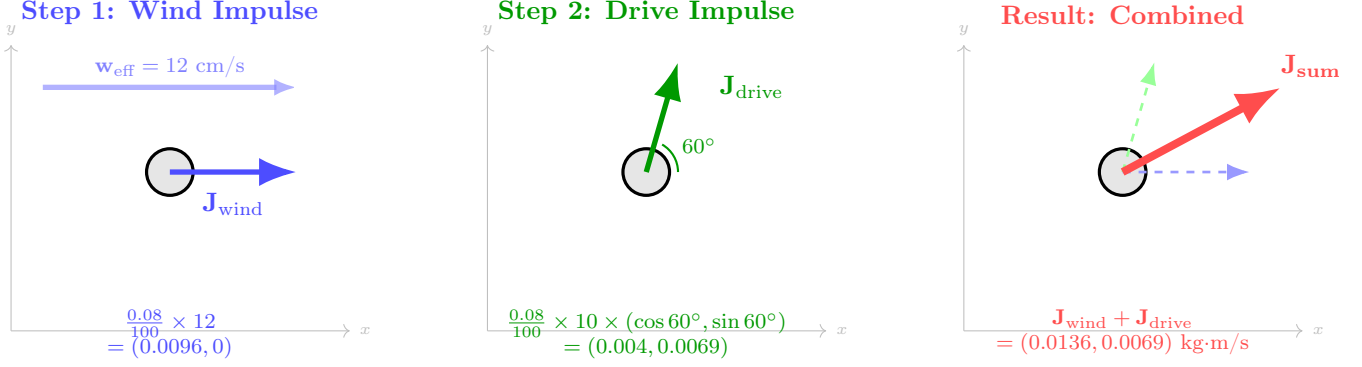


Figure 1: Impulse composition example with $m = 0.08$ kg, $v_d = 10$ cm/s, $\mathbf{w}_{\text{eff}} = 12$ cm/s, and $\psi = 60^\circ$. **Left:** wind pushes horizontally. **Middle:** robot drives at 60° . **Right:** both impulses combine (vector addition).

7.2 Wake Blocking Reduction

Problem we are solving. We need the shielding effect from an upwind blocker to fade out *smoothly* with downwind distance. A hard cutoff would flicker due to tiny motion or sensor noise, producing visible jerks. A smooth ramp avoids this.

For each upwind neighbor i , compute an overlap score

$$r_i = \exp\left(-\frac{1}{2} \left(\frac{\ell_\perp}{\sigma_\perp}\right)^2\right) \cdot \text{smoothstep}(0, L; \ell_\parallel - g), \quad (4)$$

and take the boosted, clamped maximum

$$r = \text{clamp}\left(0, 1, \gamma \cdot \max_{i \in \mathcal{N}} r_i\right), \quad \mathbf{w}_{\text{eff}} = (1 - r)\mathbf{w}.$$

Here $\mathcal{N}$ is the set of upwind neighbors; $\max_i$ means the *maximum over neighbors*.

What is smoothstep (and why use it)? $\text{smoothstep}(a, b; x)$ maps x from $[a, b]$ to a smooth ramp in $[0, 1]$:

$$\text{smoothstep}(a, b; x) = \begin{cases} 0 & x \leq a, \\ 3t^2 - 2t^3 & a < x < b, \quad t = \frac{x - a}{b - a}, \\ 1 & x \geq b. \end{cases}$$

Its slope is zero at both ends, so blocking grows and dies out gently instead of snapping on/off.

Parameter meanings (visualized in Fig. 2).

- $\ell_\parallel$: along-wind separation (downwind distance from blocker to follower, positive when follower is downwind).
- $\ell_\perp$: lateral offset from the wind centerline.
- R : blocker radius in meters (from AABB or advertised via RAB byte 0).
- $\sigma_\perp = \text{lateral_reach_radii} \cdot R$: wake width (Gaussian term).
- $L = \text{shadow_length_radii} \cdot R$: wake length (smoothstep interval).
- $g = \text{upwind_gate_radii} \cdot R$: upwind gate distance before the ramp begins.

- $\gamma = \text{gamma_boost}$: optional emphasis before clamping to $[0, 1]$.

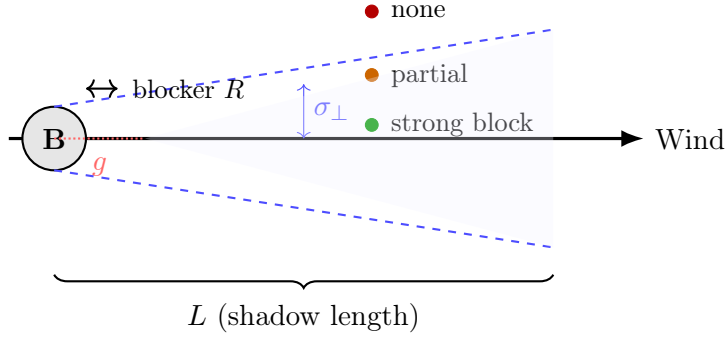


Figure 2: Blocking geometry. Adjusting $\sigma_{\perp}$ (lateral reach), L (shadow length), g (upwind gate), and γ (boost) reshapes the wake and the final reduction r .

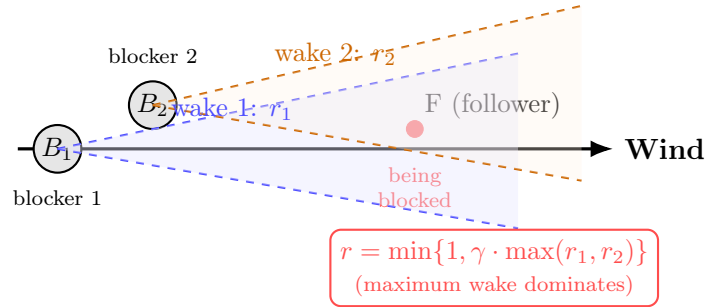


Figure 3: Two upwind blockers (B_1 and B_2) produce reductions r_1 and r_2 via (4). The final reduction is the boosted/clamped maximum.

Two-blocker illustration (max over neighbors).

8 Implementation Map (Math $\rightarrow$ Code Locations)

The equations above map directly to the base controller implementation:

- **Impulse accumulation** ($\mathbf{J}_{\text{wind}}, \mathbf{J}_{\text{drive}}, \mathbf{J}_{\text{sum}}$): `src/controllers/air_resistance/air_resistance.cpp`: `HandleAerodynamicsPreStep()`, `ApplyWindImpulse()`, `DriveImpulse()`, `HandleAerodynamicsPostStep()`.
- **Effective wind** ($\mathbf{w}_{\text{eff}} = (1-r)\mathbf{w}$): `src/controllers/air_resistance/air_resistance.cpp`: `ComputeEffectiveWind()`.
- **Wake reduction** ((4) and $r = \min(1, \gamma \cdot \max_i r_i)$): `src/controllers/air_resistance/air_resistance.cpp`: `IsBlockedByRAB(Real&)` (computes $\ell_{\parallel}$, $\ell_{\perp}$, lateral Gaussian, smoothstep fade, boost, max, clamp).
- **Tunables and units** (attributes in `<air_resistance .../>`): `src/controllers/air_resistance/air_resistance.h` (members) and `src/controllers/air_resistance/air_resistance.cpp` (read/validate in `Init()`).
- **Radius derivation** (AABB): `src/controllers/air_resistance/air_resistance.cpp`: `EnsurePhysicsHandle()`.

9 RAB Sensor: Role, Contract, Alternatives

9.1 Role

The Range-And-Bearing (RAB) medium is used as a lightweight local-neighborhood channel. This project uses it for:

- **Wake geometry:** obtain relative range/bearing to neighbors and compute upwind/lateral separations.
- **Scale exchange:** each robot advertises an effective radius in **RAB byte 0** (millimeters), so different robot models can interact without hardcoded constants.

9.2 Contract (byte-level)

- **Byte 0 is reserved** for the advertised radius (mm), written by **CAirResistance**.
- Derived/user controllers may use **bytes 1..** for their own coordination messages (IDs, roles, formation state, etc.).

9.3 Alternatives

Alternatives depend on your simulation goals:

- **Positioning sensor** could be used for global positions, but then the model depends on global knowledge (not local-only).
- **ARGoS proximity sensors** do not provide peer identity/range/bearing robustly for this wake geometry.
- **Custom medium** could encode richer state, but RAB is already standard and fast for local neighbor geometry.

10 Pseudocode: Core Functions

Variables mirror the C++ but remain language-agnostic. Distances in meters unless noted; speeds in cm/s.

10.1 Init(t_node)

```
function Init(node)
  // devices
  wheels <- GetActuator("differential_steering")
  pos_sen <- GetSensor("positioning")
  rab_sen <- GetSensor("range_and_bearing")
  rab_act <- GetActuator("range_and_bearing")

  // controller params
  v_desired_cm_s <- ReadParam(node, "velocity", default=5.0)

  // global config: <configuration><air_resistance .../>
  air <- GetConfigNode("configuration/air_resistance")
  angle_deg <- ReadAttr(air, "angle_deg")
  magnitude_cm_s <- ReadAttr(air, "magnitude")
  wind_cm_s <- magnitude_cm_s * (cos(rad(angle_deg)), sin(rad(angle_deg)))

  // optional wake/shield tunables (defaults preserved if omitted)
  lateral_reach_rad_ii <- ReadAttrDefault(air, "lateral_reach_rad_ii", 3.0)
```

```

shadow_length_radII <- ReadAttrDefault(air, "shadow_length_radII", 4.0)
gamma_boost <- ReadAttrDefault(air, "gamma_boost", 2.0)
upwind_gate_radII <- ReadAttrDefault(air, "upwind_gate_radII", 0.5)

// optional advertised-radius sanity window (meters)
adv_radius_min_m <- ReadAttrDefault(air, "adv_radius_min_m", 0.005)
adv_radius_max_m <- ReadAttrDefault(air, "adv_radius_max_m", 0.20)

// clamp obvious nonsense
lateral_reach_radII <- max(0.1, lateral_reach_radII)
shadow_length_radII <- max(0.1, shadow_length_radII)
gamma_boost <- max(1.0, gamma_boost)
upwind_gate_radII <- max(0.0, upwind_gate_radII)
adv_radius_min_m <- max(0.0001, adv_radius_min_m)
adv_radius_max_m <- max(adv_radius_min_m, adv_radius_max_m)
end

```

10.2 EnsurePhysicsHandle()

```

function EnsurePhysicsHandle()
  if body_ready: return

  entity <- GetSelfEntity()
  embodied <- entity.GetComponent("body")

  // AABB = axis-aligned bounding box
  bb <- embodied.GetBoundingBox()
  width <- bb.MaxCorner.x - bb.MinCorner.x
  depth <- bb.MaxCorner.y - bb.MinCorner.y
  self_radius_m <- 0.5 * max(width, depth)
  self_radius_m <- Clamp(self_radius_m, adv_radius_min_m, adv_radius_max_m)

  // cache dyn2d body pointer
  model <- GetDynamics2DModelForRobot()
  if model is SingleBody: body <- model.GetBody()
  else: body <- model.GetBody(index=0)
  mass_kg <- Chipmunk.GetMass(body)

  body_ready <- true
end

```

10.3 HandleAerodynamicsPreStep()

```

function HandleAerodynamicsPreStep()
  EnsurePhysicsHandle()

  J_sum <- (0, 0) // reset accumulator (world frame)
  ApplyWindImpulse() // adds (mass/100) * w_eff_cm_s

  // RAB broadcast: radius in millimeters (byte 0 reserved)
  if rab_act != null:
    r_mm <- Clamp(round(self_radius_m * 1000), 0, 255)
    rab_act.SetData(0, r_mm)
end

```

10.4 ComputeEffectiveWind()

```
function ComputeEffectiveWind()
  if Norm(wind_cm_s) < 1e-9: return wind_cm_s
  r <- 0.0
  if IsBlockedByRAB(out r):
    return max(0, 1 - r) * wind_cm_s
  return wind_cm_s
end
```

10.5 IsBlockedByRAB(out r)

```
function IsBlockedByRAB(out r)
  r <- 0.0
  if pos_sen == null or rab_sen == null: return false
  if Norm(wind_cm_s) < 1e-9: return false

  wind_dir_hat <- Normalize(wind_cm_s)
  yaw_world <- GetYawRadians()

  any <- false
  for msg in rab_sen.GetReadings():

    range_m <- 0.01 * msg.Range // cm -> m
    if range_m <= 1e-6: continue

    // blocker radius from msg.Data[0] (mm->m) else fallback
    blocker_r_m <- self_radius_m
    if Size(msg.Data) >= 1:
      adv_m <- 0.001 * msg.Data[0]
      if adv_radius_min_m < adv_m and adv_m < adv_radius_max_m:
        blocker_r_m <- adv_m

    bearing_world <- yaw_world + msg.HorizontalBearing

    me_to_other <- (range_m * cos(bearing_world),
                  range_m * sin(bearing_world))
    other_to_me <- -me_to_other

    along_m <- Dot(other_to_me, wind_dir_hat) // + means neighbor is upwind
    gate_m <- max(1e-6, upwind_gate_radii * blocker_r_m)
    if along_m <= gate_m: continue

    lateral_vec <- other_to_me - along_m * wind_dir_hat
    lateral_m <- Norm(lateral_vec)

    sigma <- lateral_reach_radii * blocker_r_m
    L <- shadow_length_radii * blocker_r_m

    lateral_cov <- exp(-0.5 * (lateral_m / max(1e-6, sigma))^2)
    fade <- smoothstep(0, L, along_m - gate_m)
    red_i <- lateral_cov * fade

    // emphasize mid-range overlaps (optional), then clamp via max+final clamp
    red_i <- 1 - (1 - red_i) ^ gamma_boost

  r <- max(r, red_i)
```

```

    any <- true

    r <- min(1.0, r)
    return any and (r > 1e-6)
end

```

10.6 DriveImpulse(v_cm_s)

```

function DriveImpulse(v_cm_s)
  yaw <- GetYawRadians()
  fwd <- (cos(yaw), sin(yaw)) // unit forward vector (world frame)
  J_sum += (mass_kg/100.0) * v_cm_s * fwd
end

```

10.7 ApplyWindImpulse()

```

function ApplyWindImpulse()
  w_eff_cm_s <- ComputeEffectiveWind()
  if Norm(w_eff_cm_s) < 1e-9: return
  J_sum += (mass_kg/100.0) * w_eff_cm_s
end

```

10.8 HandleAerodynamicsPostStep()

```

function HandleAerodynamicsPostStep()
  space <- Chipmunk.Space(body)
  Chipmunk.AddPostStepCallback(space, lambda:
    Chipmunk.ApplyImpulseAtWorldPoint(body, J_sum, position=COM))
end

```

10.9 GetYawRadians()

```

function GetYawRadians()
  q <- pos_sen.GetReading().Orientation()
  return q.Yaw()
end

```

11 Tuning and Debugging

Key tunables (typical defaults)

- lateral_reach_radial widens or narrows the Gaussian cross-section.
- shadow_length_radial lengthens the downwind extent.
- upwind_gate_radial ignores near side-by-side neighbors.
- gamma_boost emphasizes mid-range overlaps before clamping.

Debug tips

- Print intermediate terms inside the blocking loop: $\ell_{\parallel}$, $\ell_{\perp}$, r_i , and final r .
- Verify the on-screen wind arrow matches XML (`angle_deg`, `magnitude`).
- If a robot seems unaffected, check advertised radius byte conversion ($\text{mm} \rightarrow \text{m}$), AABB-derived R , and that neighbors are truly upwind ($\ell_{\parallel} > 0$).

12 Doxygen API Documentation

12.1 What Doxygen is for

Doxygen generates browsable API documentation (HTML) from C++ headers/sources and special comment blocks.

12.2 How to rebuild

From the repository root:

```
doxygen doc/Doxyfile
```

This generates HTML output under `doc/html/`. The folder `doc/html/` should be **gitignored** (generated artifact).

12.3 How to view

Open `doc/html/index.html` in a browser.

References

- [1] D. H. Stolfi and G. Danoy, “Design and analysis of an E-Puck2 robot plug-in for the ARGoS simulator,” *Robotics and Autonomous Systems*, vol. 164, p. 104412, 2023. doi: 10.1016/j.robot.2023.104412.